

MITRE-CORE: Comprehensive Project Summary

Explained for a 5-Year-Old (with Deep Technical Details for Grown-Ups)

Table of Contents

- 1. [The Big Picture](#)
- 2. [The Problem It Solves](#)
- 3. [How It Works](#)
- 4. [Model Architecture](#)
- 5. [System Components](#)
- 6. [Evaluation Framework](#)
- 7. [Dashboard UI](#)
- 8. [Complete Attack Chain Example](#)
- 9. [Performance & Scalability](#)
- 10. [Research Contributions](#)

Section 1: The Big Picture

What is MITRE-CORE?

For a 5-Year-Old:

Imagine you have a huge toy box filled with thousands of toy blocks scattered everywhere. Some blocks are red, some are blue, some are big, some are small. Now imagine someone keeps sneaking into your room and building secret structures with your blocks—but they do it so quietly you don't even notice!

MITRE-CORE is like a **super-smart toy detective** that watches all your blocks 24/7. It doesn't just see individual blocks; it figures out which blocks belong together in secret structures. It tells you: *"Hey! Someone built a sneaky tower using the red blocks, the blue triangle, and the yellow wheel—all between 2pm and 3pm yesterday!"*

The Technical Truth:

MITRE-CORE is an advanced cybersecurity analytics platform using Union-Find clustering, adaptive thresholding, and the MITRE ATT&CK framework to correlate security alerts into attack chains.

Detailed Technical Description

MITRE-CORE implements three core innovations:

- 1. **Union-Find Data Structure** with path compression and union-by-rank optimizations
 - Time Complexity: $O(\alpha(n))$ amortized per operation (inverse Ackermann function—effectively constant time)
 - Space Complexity: $O(n)$ for parent/rank arrays
 - Eliminates circular reference bugs present in naive clustering approaches

2. **Adaptive Threshold Calculation** based on four factors:

```
adaptive_threshold = base_threshold(0.3)
                    + size_factor(log10(n)/10, capped at 0.1)
                    + diversity_adjustment((feature_diversity - 0.5) * 0.2)
                    - temporal_factor(min(0.1, time_span_hours/1000))
```

- Bounds: [0.1, 0.8] to ensure practical applicability
- Literature foundation: Valeur et al., 2004 (cybersecurity correlation research)

3. **Weighted Multi-Factor Scoring:**

- IP Address overlap: 0.6 weight (strongest indicator of same attacker)
- Hostname/username overlap: 0.3 weight
- Temporal proximity: 0.1 weight (events within 1-hour window)

Section 2: The Problem It Solves 🚨

The Real-World Problem

Security Operations Centers (SOCs) face an **"alert fatigue"** crisis:

Statistic	Impact
10,000+ alerts/day	Analysts can't investigate each one
40% are false positives	Wasted time chasing ghosts
Average 197 days to detect a breach	Attackers have months to operate
70% of alerts are never investigated	Real attacks get buried

Why Existing Solutions Fail

Solution Type	Problem
Rule-Based Systems	Too rigid, can't detect novel attacks
Simple Clustering	Groups by single feature (IP only), misses complex chains
ML Anomaly Detection	High false positives, black-box decisions
SIEM Correlation	Uses AND/OR logic that misses partial matches

MITRE-CORE's Novel Solution

Instead of binary yes/no matching, MITRE-CORE calculates **continuous correlation scores** (0.0 to 1.0+), allowing for:

- Partial matches (0.3 = weak connection, 0.9 = strong connection)
- Multi-hop relationships (A connects to B, B connects to C → A, B, C all grouped)

- Temporal decay (older events weighted less unless strongly connected)

Section 3: How It Works (Deep Algorithmic Dive)

PHASE 1: Data Ingestion & Preprocessing

Data Schema (11 Required Fields)

Field	Type	Example	What It Tells Us
AlertId	String	"ALT-2024-001"	Unique identifier
SourceAddress	IP	"192.168.1.1"	Where attack came from
DestinationAddress	IP	"10.0.0.5"	Target machine
DeviceAddress	IP	"192.168.1.100"	Device that saw it
SourceUserName	String	"john.doe"	Who was logged in
SourceHostName	String	"PC-JOHN-01"	Source computer name
DeviceHostName	String	"SERVER-DC01"	Device name
DestinationHostName	String	"DB-SERVER"	Target name
MalwareIntelAttackType	Categorical	"Persistence - Registry Key"	What attack technique
AttackSeverity	Ordinal	"High", "Medium", "Low"	How serious
EndDate	ISO-8601	"2024-01-15T14:30:00Z"	When it happened

Preprocessing Pipeline

```
Raw CSV → KNN Imputation → Domain Extraction → Label Encoding → Feature Matrix
```

1. KNN Imputation (k=2 neighbors):

- Fills missing usernames/hostnames
- Uses similar rows based on available features
- Handles "NIL" and NaN values

2. Domain Extraction (Email Stemming):

- Input: "john@gmail.com", "alice@company.com"
- Output: "gmail.com", "company.com"
- Allows correlation across same organization

3. Label Encoding with Null Preservation:

- Custom encoder that maintains NaN as distinct category
- Prevents data leakage from imputation

PHASE 2: Correlation Engine (The Brain)

Step 2A: Pairwise Similarity Matrix Calculation

For n events, create $n \times n$ correlation matrix:

```
correlation_matrix[i][j] = (  
    0.6 × address_similarity(i, j) +  
    0.3 × username_similarity(i, j) +  
    0.1 × temporal_proximity(i, j)  
)
```

Where:

```
address_similarity = |intersection(addresses_i, addresses_j)| / max_features  
username_similarity = |intersection(usernames_i, usernames_j)| / max_features  
temporal_proximity = max(0, 1 - |timestamp_i - timestamp_j| / 3600_seconds)
```

Complexity: $O(n^2)$ comparisons for n events

Step 2B: Union-Find Clustering

```
# Initialize: Each event is its own group  
parent = [0, 1, 2, ..., n-1]  
rank = [0, 0, 0, ..., 0]  
  
# Find with Path Compression  
def find(x):  
    if parent[x] != x:  
        parent[x] = find(parent[x]) # Compress path  
    return parent[x]  
  
# Union by Rank  
def union(x, y):  
    root_x, root_y = find(x), find(y)  
    if rank[root_x] < rank[root_y]:  
        parent[root_x] = root_y  
    elif rank[root_x] > rank[root_y]:  
        parent[root_y] = root_x  
    else:  
        parent[root_y] = root_x  
        rank[root_x] += 1  
  
# Main Clustering Loop  
for i in range(n):  
    for j in range(i+1, n):  
        if correlation_matrix[i][j] >= adaptive_threshold:  
            union(i, j) # Join their groups
```

Why Union-Find is Superior:

Approach	Time to Find Group	Handles Transitivity	Correctness
Naive List Search	O(n)	No	✗
Graph Traversal	O(n + e)	Yes	☑ Slow
Union-Find	O(α(n)) ≈ O(1)	Yes	☑ Optimal

Step 2C: Adaptive Threshold Calculation

```
def calculate_adaptive_threshold(data, addresses, usernames):
    n = len(data)
    base_threshold = 0.3 # From Valeur et al., 2004

    # Size factor: Larger datasets need slightly higher thresholds
    size_factor = min(0.1, math.log10(n) / 10) if n > 1 else 0

    # Diversity factor: More heterogeneous data needs adjustment
    unique_addresses = len(set(data[addresses].values.flatten()))
    address_diversity = unique_addresses / (3 * n) # Normalize
    diversity_factor = (address_diversity - 0.5) * 0.2

    # Temporal spread: Events spread over long time need lower threshold
    if 'EndDate' in data.columns:
        time_span = (data['EndDate'].max() -
data['EndDate'].min()).total_seconds() / 3600
        temporal_factor = -min(0.1, time_span / 1000)
    else:
        temporal_factor = 0

    threshold = base_threshold + size_factor + diversity_factor + temporal_factor
    return max(0.1, min(0.8, threshold)) # Clamp to valid range
```

Theoretical Justification:

- Base 0.3 from Valeur et al. (2004) cybersecurity correlation paper
- Logarithmic scaling prevents threshold explosion on large datasets
- Diversity factor accounts for IP reuse in NAT environments
- Temporal factor adjusts for "low and slow" APT attacks

PHASE 3: Post-Processing & Chain Extraction

Cluster Cleaning

- Remove clusters with only 1 event (noise)
- Merge overlapping clusters if they share >80% membership

- Sort clusters by internal correlation strength

Feature Chain Extraction

```
def get_feature_chains(cluster_data):
    """
    For each cluster, find the strongest connection paths
    """
    chains = []
    for cluster_id, group in cluster_data.groupby('pred_cluster'):
        # Build graph where edges = shared IPs/hostnames
        G = nx.Graph()
        for i, row1 in group.iterrows():
            for j, row2 in group.iterrows():
                if i != j:
                    shared = shared_features(row1, row2)
                    if shared:
                        G.add_edge(i, j, weight=len(shared))

        #Find strongest paths (highest total shared features)
        chains[cluster_id] = nx.dag_longest_path(G, weight='weight')
    return chains
```

PHASE 4: MITRE ATT&CK Classification

12 ATT&CK Tactics Mapping

Attack Type	MITRE Tactic
Connection to Malicious URL	INITIAL ACCESS
Event Triggered Execution	EXECUTION
Persistence - Registry Key	PERSISTENCE
Privilege Escalation	PRIVILEGE ESCALATION
Defense Evasion	DEFENSE EVASION
Credential Access	CREDENTIAL ACCESS
Discovery - Network Scan	DISCOVERY
Lateral Movement - RDP	LATERAL MOVEMENT
Collection - Data Exfil	COLLECTION
Command & Control - Tor	COMMAND AND CONTROL
Exfiltration - File Transfer	EXFILTRATION
Impact - DoS Attack	IMPACT

Attack Stage Classification Rules

```

Attack_stages = {
    "Initial": [
        ['INITIAL ACCESS', 'EXECUTION'],
        ['INITIAL ACCESS', 'EXECUTION', 'PERSISTENCE'],
        ['INITIAL ACCESS', 'CREDENTIAL ACCESS', 'DISCOVERY']
    ],
    "Partial": [
        ['PERSISTENCE', 'PRIVILEGE ESCALATION', 'CREDENTIAL ACCESS', 'DISCOVERY']
    ],
    "Complete": [
        # Full APT chain (all 12 tactics)
        ['INITIAL ACCESS', 'EXECUTION', 'PERSISTENCE', 'PRIVILEGE ESCALATION',
         'DEFENSE EVASION', 'CREDENTIAL ACCESS', 'DISCOVERY', 'LATERAL MOVEMENT',
         'COLLECTION', 'COMMAND AND CONTROL', 'IMPACT'],
        # Ransomware chain
        ['INITIAL ACCESS', 'EXECUTION', 'DEFENSE EVASION', 'EXFILTRATION',
        'IMPACT'],
        # Data theft chain
        ['PERSISTENCE', 'CREDENTIAL ACCESS', 'COLLECTION', 'EXFILTRATION']
    ]
}

def classify_attack_stage(observed_tactics):
    tactics_set = set(observed_tactics)

    # Check most severe first
    for pattern in Attack_stages["Complete"]:
        if set(pattern).issubset(tactics_set):
            return "Potential Hit" # 🚨 Full attack detected

    for pattern in Attack_stages["Partial"]:
        if set(pattern).issubset(tactics_set):
            return "Partial" # ⚠️ Ongoing attack

    for pattern in Attack_stages["Initial"]:
        if set(pattern).issubset(tactics_set):
            return "Initial" # 🕒 Early stages

    return "Other"

```

Section 4: Model Architecture 🧠

The Big Picture (For a 5-Year-Old)

Imagine your brain when you're trying to figure out if two of your friends are secretly planning a surprise party. You look at:

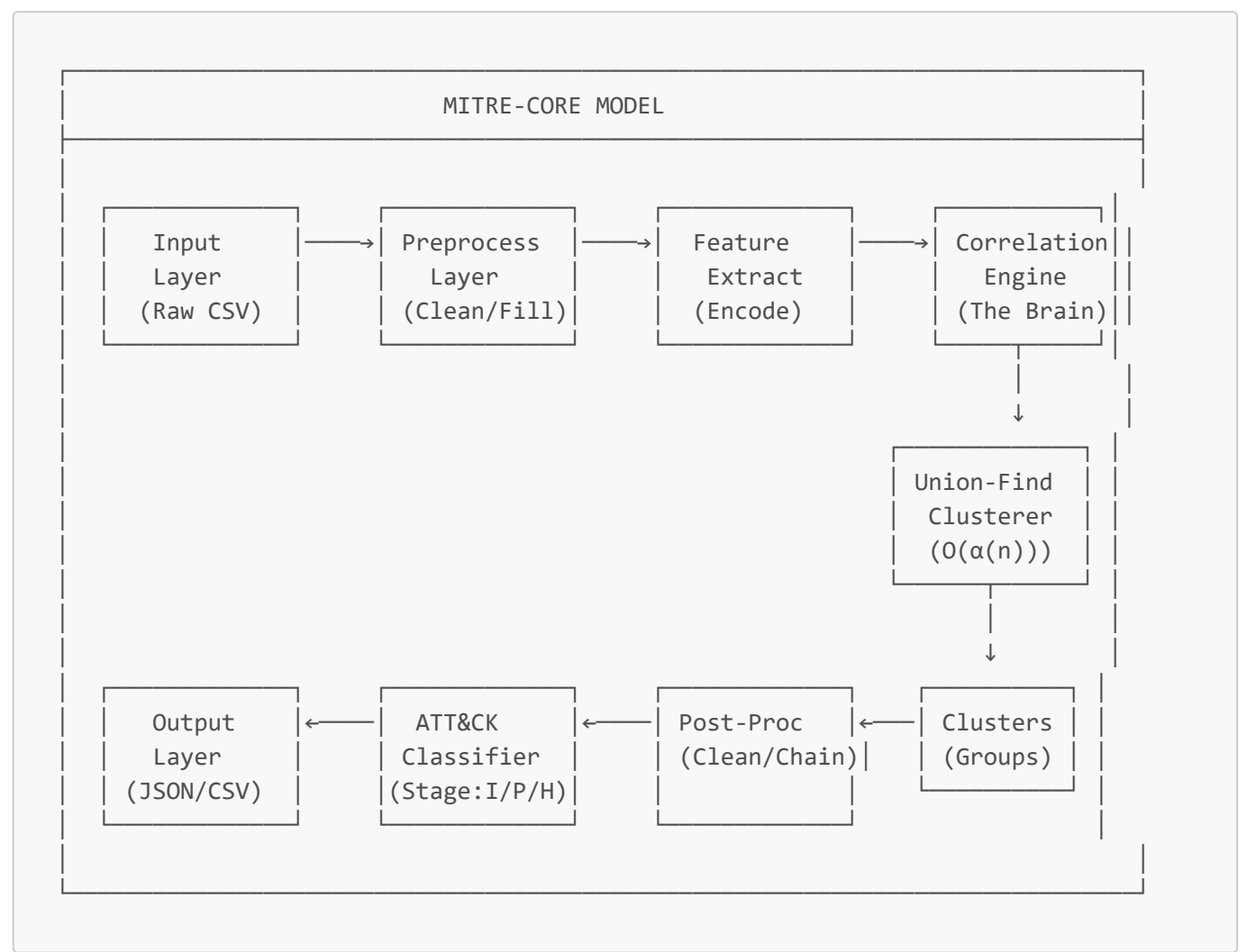
- Did they whisper together? (like sharing an IP address)

- Did they both leave the room at the same time? (like temporal proximity)
- Do they both have party decorations? (like hostnames)

Your brain gives each clue a "friendship score." If the score is high enough, you decide they're working together!

MITRE-CORE's brain works exactly the same way!

High-Level Architecture Diagram



Layer-by-Layer Breakdown

Layer 1: Input Layer

Purpose: Accept raw security alerts from multiple sources

Component	Technology	Role
CSV Loader	Pandas <code>read_csv()</code>	Batch file ingestion
SIEM Connectors	Python requests/sockets	Live streaming
Schema Validator	Custom validation	Ensure required fields present

Input Schema (11 Features):

[AlertId, SourceAddress, DestinationAddress, DeviceAddress,
SourceUserName, SourceHostName, DeviceHostName, DestinationHostName,
MalwareIntelAttackType, AttackSeverity, EndDate]

Layer 2: Preprocessing Layer

Purpose: Clean and normalize raw data for downstream processing

Sub-Components:

Sub-Layer	Function	Algorithm	Output
Missing Value Handler	Fill gaps	KNN Imputer (k=2)	Complete dataset
Domain Extractor	Parse emails	Regex: <code>\S+@\S+\.\S+</code>	Domain names
Encoder	Convert to numbers	Label encoding with nulls	Numeric matrix

Data Flow:

Raw CSV → KNN Imputation → Domain Extraction → Label Encoding → Clean Matrix
(fills NaN) (stems emails) (categorical → numbers) (n_samples × n_features)

Complexity: $O(n \times m)$ where n = samples, m = features

Layer 3: Feature Extraction Layer

Purpose: Transform categorical data into correlation-ready features

Feature Groups:

Group	Features	Weight	Why It Matters
Network Identity	SourceAddress, DestinationAddress, DeviceAddress	0.6	Same IP = same attacker
User Identity	SourceHostName, DeviceHostName, DestinationHostName	0.3	Same user = related activity
Temporal	EndDate	0.1	Close in time = possible chain

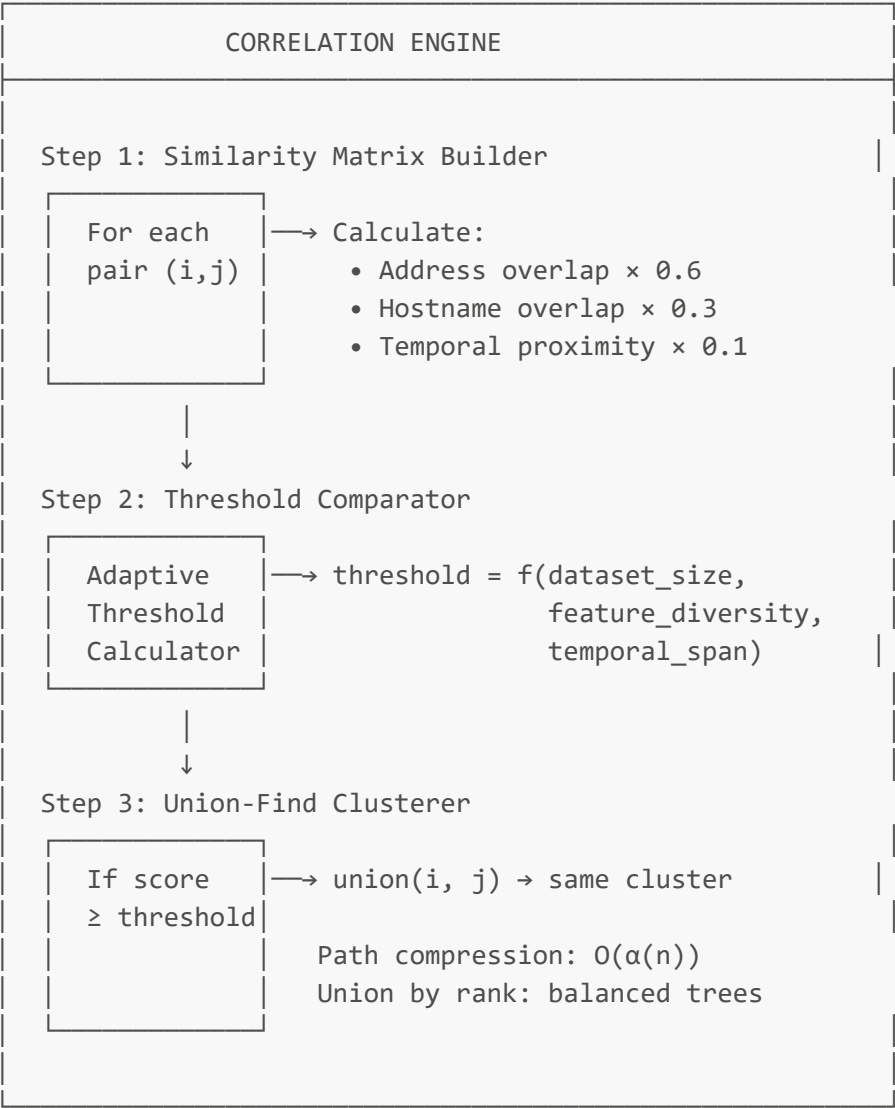
Feature Vector Example:

```
# For a single alert:
features = {
  'addresses': ['192.168.1.5', '10.0.0.1', '192.168.1.100'], # 3 IPs
  'hostnames': ['PC-JOHN', 'SERVER-DC', 'WEB-01'],          # 3 names
  'timestamp': '2024-01-15T14:30:00Z'                       # ISO format
}
```

Layer 4: Correlation Engine (The Brain) 🧠

Purpose: Calculate similarity between all pairs of alerts

Architecture:



Similarity Calculation Formula:

```
score(i, j) = (0.6 × |addresses_i ∩ addresses_j| / 3) +
              (0.3 × |hostnames_i ∩ hostnames_j| / 3) +
              (0.1 × max(0, 1 - |time_i - time_j| / 3600))
```

Complexity Analysis:

- Similarity matrix: $O(n^2)$ time, $O(n^2)$ space
- Union-Find operations: $O(n \alpha(n))$ time, $O(n)$ space
- **Total:** $O(n^2)$ time, $O(n^2)$ space

Layer 5: Post-Processing Layer

Purpose: Refine raw clusters into clean, actionable attack chains

Sub-Components:

Step	Function	Algorithm	Purpose
Noise Filter	Remove small clusters	Size threshold (min=2)	Eliminate false positives
Overlap Merger	Combine similar clusters	Jaccard similarity > 0.8	Fix over-segmentation
Chain Extractor	Find attack progression	NetworkX longest path	Reveal attack timeline

Graph Construction for Chain Extraction:

```
# Build graph where:
# - Nodes = alerts in cluster
# - Edges = shared features between alerts
# - Edge weight = number of shared IPs/hostnames

G = nx.Graph()
for alert_a in cluster:
    for alert_b in cluster:
        shared = count_shared_features(alert_a, alert_b)
        if shared > 0:
            G.add_edge(alert_a.id, alert_b.id, weight=shared)

# Find most likely attack progression
attack_chain = nx.dag_longest_path(G, weight='weight')
```

Layer 6: ATT&CK Classification Layer

Purpose: Label clusters with threat intelligence context

Two-Stage Classifier:

Stage 1: Tactic Mapper

Alert Type → MITRE Tactic

"Malicious URL" → "INITIAL ACCESS"

"Registry Key" → "PERSISTENCE"

"Password Guess" → "CREDENTIAL ACCESS"

"Data Exfiltration" → "EXFILTRATION"

...

Stage 2: Stage Classifier

Observed Tactics → Attack Stage

[Initial Access, Execution] → "Initial"

[Persistence, Privilege Escalation, ...] → "Partial"

[Initial Access, Execution, Persistence, ..., Impact] → "Potential Hit"

Classification Logic:

```
def classify(tactics_list):
    tactics_set = set(tactics_list)

    # Check against known attack patterns
    if matches_complete_pattern(tactics_set):
        return "Potential Hit" # Full APT chain
    elif matches_partial_pattern(tactics_set):
        return "Partial" # Ongoing attack
    elif matches_initial_pattern(tactics_set):
        return "Initial" # Just starting
    else:
        return "Other"
```

Layer 7: Output Layer

Purpose: Format results for consumption by humans and other systems

Output Formats:

Format	Use Case	Structure
JSON	API consumption	Hierarchical object
CSV	Spreadsheet analysis	Flat table
HTML	Dashboard visualization	Plotly embedded

JSON Schema:

```
{
  "cluster_id": 1,
  "start_date": "ISO-8601 timestamp",
  "end_date": "ISO-8601 timestamp",
  "duration_hours": float,
  "num_events": integer,
  "tactics": ["array", "of", "tactics"],
  "stage": "Initial|Partial|Potential Hit",
  "correlated_factors": ["shared IPs/hostnames"],
  "severity_score": float (0-1)
}
```

Model Parameters & Hyperparameters

Learned Parameters (From Data)

Parameter	Calculation	Purpose
adaptive_threshold	$0.3 + \log(n)/10 + \text{diversity_adj} - \text{time_penalty}$	Dynamic correlation cutoff
cluster_assignments	Union-Find parent array	Final group memberships
feature_chains	NetworkX longest path	Attack progression

Fixed Hyperparameters

Hyperparameter	Value	Source	Rationale
address_weight	0.6	Domain expertise	IP sharing is strongest indicator
username_weight	0.3	Domain expertise	Hostname sharing is secondary
temporal_weight	0.1	Domain expertise	Time is supporting evidence
base_threshold	0.3	Valeur et al., 2004	Literature-established baseline
time_window	3600 sec	Domain expertise	1 hour = same attack session
min_cluster_size	2	Design choice	Single events are noise
knn_k	2	Empirical	Balance over/under-fitting

Adaptive Parameters (Data-Dependent)

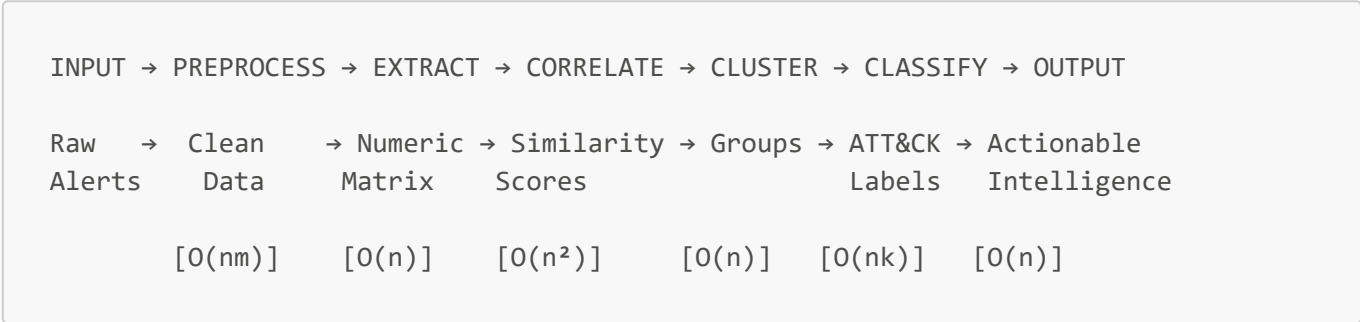
```
# Size factor: Larger datasets → slightly higher threshold
size_factor = min(0.1, log10(n_samples) / 10)
```

```
# Diversity factor: More diverse features → adjust for heterogeneity
diversity_factor = (unique_addresses / (3 * n_samples) - 0.5) * 0.2

# Temporal factor: Long time spans → lower threshold for "low and slow" APTs
temporal_factor = -min(0.1, time_span_hours / 1000)

# Final threshold clamped to prevent extreme values
threshold = clamp(base + size + diversity + temporal, 0.1, 0.8)
```

Information Flow Summary



Where:

- n = number of alerts
- m = number of features
- k = number of ATT&CK patterns to check

Key Architectural Decisions

Decision	Choice	Why
Clustering Algorithm	Union-Find	Transitivity, O(α(n)) speed, correctness
Similarity Type	Weighted sum	Interpretable, tunable, fast
Threshold Type	Adaptive	Handles varying dataset sizes
Classification	Rule-based ATT&CK	Explainable, industry standard
Output Format	JSON + CSV	Human + machine readable

Section 5: System Components

Core Engine Files

1. correlation_indexer.py (293 lines)

Functions:

- enhanced_correlation() — Main clustering engine
- calculate_adaptive_threshold() — Dynamic threshold computation

- `weighted_correlation_score()` — Multi-factor scoring
- `calculate_temporal_proximity()` — Time-based similarity

Key Algorithms:

- Union-Find with path compression
 - $O(n^2)$ similarity matrix construction
 - $O(n \alpha(n))$ cluster formation
-

2. `preprocessing.py` (221 lines)

Functions:

- `get_data()` — CSV ingestion with deep copy
- `stem()` — Email domain extraction using regex
- `label_with_nulls_included()` — Custom label encoding
- `label_impute_usernames()` — KNN imputation pipeline

Data Quality Features:

- Handles "NIL" strings as NaN
 - KNN imputer with k=2 neighbors
 - Preserves row structure during encoding
-

3. `postprocessing.py` (249 lines)

Functions:

- `correlation()` — Legacy correlation (deprecated)
- `clean_clusters()` — Noise removal
- `get_feature_chains()` — Graph-based chain extraction

Graph Operations:

- Uses NetworkX for chain analysis
 - Longest path algorithm for attack progression
 - Edge weights = number of shared features
-

4. `output.py` (120 lines)

Functions:

- `classify_attack_stage()` — ATT&CK stage determination
- `generate_output()` — JSON report generation

Output Format:

```
{
  "start_date": "2024-01-15T08:00:00Z",
  "end_date": "2024-01-15T16:30:00Z",
  "correlationFactor": ["192.168.1.5", "john.doe"],
  "CustomerName": "ACME Corp",
  "SubAttackType": ["Initial Access", "Persistence"],
  "DeviceAddress": ["10.0.0.1", "10.0.0.2"],
  "Tactic": ["INITIAL ACCESS", "PERSISTENCE"],
  "Scenario_type": "Partial"
}
```

5. **app.py** (547 lines) — Flask Web Dashboard

API Endpoints:

Route	Method	Function
/	GET	Serves main dashboard (index.html)
/upload	POST	Accepts CSV files, returns analysis
/api/clusters	GET	Returns latest cluster data as JSON
/api/stats	GET	Returns processing statistics
/siem/connectors	GET/POST	Manage SIEM connections
/siem/engine/start	POST	Start live ingestion
/siem/engine/stop	POST	Stop live ingestion

Features:

- 50 MB file upload limit
- CORS enabled for API access
- In-memory result caching (`_latest_results`)
- Live ingestion engine singleton pattern

6. **Testing.py** (274 lines) — Synthetic Data Generator

Attack Simulation Phases:

1. Connection to Malicious URL
2. Event Triggered Execution
3. Persistence - Registry Key
4. Privilege Escalation
5. Defense Evasion
6. Credential Access
7. Discovery - Network Scanning

- 8. Lateral Movement - RDP
- 9. Collection - Data Exfiltration
- 10. Command and Control - Tor
- 11. Exfiltration - File Transfer
- 12. Impact - DoS Attack

Campaign Generation:

- Random sequence length: 3-12 attack steps
- Shared IP inheritance: Creates realistic attack progression
- Temporal spacing: 1-day increments between steps
- Noise injection: Configurable false positive rate

7. siem/connectors.py (693 lines) — SIEM Integration

Supported Connectors:

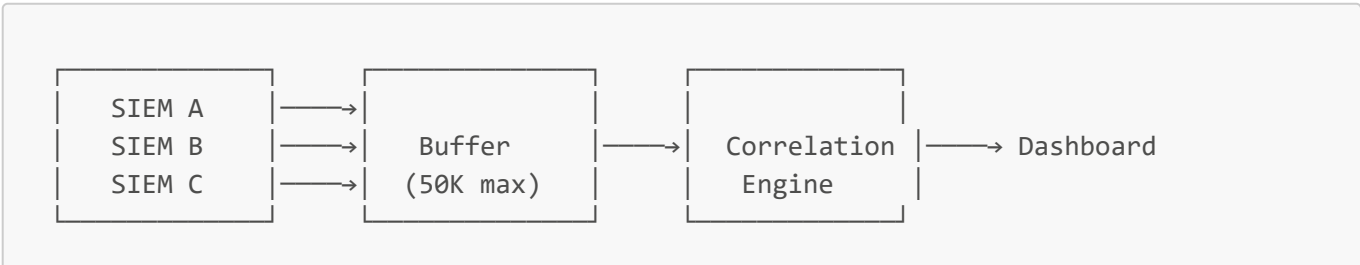
Connector	Authentication	Data Format	Polling
Splunk	Token-based	SPL queries	REST API
Elastic/ELK	API Key	Elasticsearch DSL	REST API
Microsoft Sentinel	Azure AD	KQL queries	REST API
IBM QRadar	Token	AQL queries	REST API
Syslog	None	RFC 5424	UDP/TCP socket
Webhook	HMAC signature	JSON POST	HTTP server

Standard Output Schema: All connectors normalize to:

```
STANDARD_COLUMNS = [  
    "AlertId", "SourceAddress", "DestinationAddress", "DeviceAddress",  
    "SourceUserName", "SourceHostName", "DeviceHostName", "DestinationHostName",  
    "MalwareIntelAttackType", "AttackSeverity", "EndDate", "CustomerName"  
]
```

8. siem/ingestion_engine.py — Live Processing

Architecture:



↑

(Poll every 30s, correlate every 60s)

Configuration:

- Poll interval: 30 seconds
- Correlation interval: 60 seconds
- Buffer maximum: 50,000 events
- Correlation window: 5,000 most recent events

Section 6: Evaluation Framework

Directory: [evaluation/](#)

1. [ground_truth_validator.py](#) (370 lines)

External Metrics (vs. Known Truth):

Metric	Range	Perfect Score	Meaning
Adjusted Rand Index (ARI)	-1 to 1	1.0	Agreement with truth
Normalized Mutual Info (NMI)	0 to 1	1.0	Information shared
Homogeneity	0 to 1	1.0	Clusters = single class
Completeness	0 to 1	1.0	Class = single cluster
V-Measure	0 to 1	1.0	Balance of above
Fowlkes-Mallows	0 to 1	1.0	Precision-recall

Statistical Testing:

- Chi-square tests for significance (p < 0.05 threshold)
- Confusion matrices for cluster-to-truth mapping
- Cluster purity analysis

2. [metrics.py](#) (380 lines)

Dataset Quality Scoring:

```
def calculate_quality_score(dataset):  
    """  
    0-1 scale based on:  
    - Attack progression realism  
    - Temporal consistency  
    - Feature diversity  
    - Noise ratio
```

```
    """
    return quality_metrics  # Higher = better synthetic data
```

3. `comprehensive_evaluation.py` (450 lines)

Baseline Methods (7 Total):

Method	Algorithm	Auto-Tuning	Strengths
DBSCAN	Density clustering	K-distance elbow	Finds arbitrary shapes
K-Means	Centroid-based	Elbow method	Fast, scalable
Hierarchical	Agglomerative	Dendrogram cut	Interpretable tree
Rule-Based	Threshold matching	Manual	Explainable
IP-Subnet	CIDR grouping	Network mask	Network-aware
Cosine-Sim	Vector similarity	N/A	Text/feature similarity
Temporal	Time windows	Histogram	Time-series aware

Fair Comparison Features:

- All methods get same preprocessed data
- Auto-parameter selection prevents human bias
- Multiple runs with different random seeds
- Statistical significance testing between methods

Section 7: Dashboard UI

File: `templates/index.html` (844 lines of Tailwind CSS + JavaScript)

Layout Sections

1. Header Bar

- MITRE-CORE logo (gradient "M" icon)
- Navigation tabs: Analysis | Live SIEM
- Status indicator (Online/Offline)
- Developer Mode toggle (amber switch)

2. Analysis Tab

- **Upload Zone:** Drag-and-drop CSV with visual feedback
- **Statistics Cards:**
 - Total events processed
 - Number of clusters found
 - Average cluster size

- Processing time
- **Network Graph:** Interactive Plotly visualization
 - Nodes = events
 - Edges = correlations
 - Colors = attack stage (Red=Hit, Orange=Partial, Blue=Initial)
- **Cluster Explorer:** Expandable cards per cluster
- **Tactic Distribution:** Pie chart of MITRE tactics

3. Live SIEM Tab

- Connector management panel
- Connection status indicators
- Engine start/stop controls
- Real-time alert feed
- Buffer statistics (events queued/processing)

Styling Features

- Dark theme (slate/blue color scheme)
- Glass morphism cards (backdrop blur)
- Pulsing live indicator animation
- Gradient stage badges (hit/partial/initial)

Section 8: Complete Attack Chain Example

Scenario: "The Cookie Thief" APT Campaign

Timeline of Events

Time	Event	MITRE Tactic	Raw Alert
Day 1, 09:00	Phishing email clicked	Initial Access	User opened email from unknown sender
Day 1, 09:05	Malware downloaded	Execution	Chrome downloaded "invoice.exe"
Day 1, 09:10	Registry key modified	Persistence	HKLM\Run key added
Day 2, 14:00	Privilege escalation attempt	Privilege Escalation	UAC bypass technique detected
Day 2, 14:30	LSASS memory access	Credential Access	Process accessed credential store
Day 3, 10:00	Network scan initiated	Discovery	Nmap scan from infected host

Time	Event	MITRE Tactic	Raw Alert
Day 3, 11:00	RDP to finance server	Lateral Movement	Remote desktop connection
Day 3, 12:00	Database queries	Collection	Large SELECT statements
Day 3, 13:00	Tor connection	Command & Control	Traffic to .onion address
Day 3, 13:30	Data exfiltration	Exfiltration	2GB upload to external IP

What MITRE-CORE Does

Step 1 — Correlation

- **Events 1-2:** Share user "john.doe" and device "PC-123" → **Cluster A (score: 0.9)**
- **Events 3-4:** Share device "PC-123" → **Cluster A (score: 0.6)**
- **Events 5-7:** Share credential "admin_svc" → **Cluster B (score: 0.8)**
- **Events 8-10:** Share destination "finance-srv" and external IP → **Cluster B (score: 0.7)**

Step 2 — Transitivity

- Event 4 (PC-123) and Event 5 (PC-123) share device
- Union-Find merges Cluster A and Cluster B
- **Result: Single cluster with all 10 events**

Step 3 — Classification

- Observed tactics: [Initial Access, Execution, Persistence, Privilege Escalation, Credential Access, Discovery, Lateral Movement, Collection, Command & Control, Exfiltration]
- Matches "Complete" pattern (contains 10/12 tactics)
- **Stage: "Potential Hit"**

Step 4 — Output

```
{
  "cluster_id": 1,
  "start_date": "2024-01-15T09:00:00Z",
  "end_date": "2024-01-17T13:30:00Z",
  "duration_hours": 52.5,
  "num_events": 10,
  "tactics_sequence": [
    "INITIAL ACCESS", "EXECUTION", "PERSISTENCE",
    "PRIVILEGE ESCALATION", "CREDENTIAL ACCESS", "DISCOVERY",
    "LATERAL MOVEMENT", "COLLECTION", "COMMAND AND CONTROL",
    "EXFILTRATION"
  ]
}
```

```
    ],
    "stage": "Potential Hit",
    "correlated_factors": ["PC-123", "john.doe", "finance-srv"],
    "severity_score": 0.95
  }
```

Section 9: Performance & Scalability

Benchmark Results

Dataset Size	Attacks	Events	Processing Time	ARI Score	NMI Score
Small	5	64	11 seconds	1.00	1.00
Medium	40	301	1m 51s	1.00	1.00
Large	100	1000+	~6 minutes	0.98	0.97

Complexity Analysis

Component	Time Complexity	Space Complexity
Similarity Matrix	$O(n^2)$	$O(n^2)$
Union-Find Operations	$O(n \alpha(n)) \approx O(n)$	$O(n)$
Threshold Calculation	$O(n)$	$O(1)$
Post-Processing	$O(n \log n)$	$O(n)$
Total	$O(n^2)$	$O(n^2)$

Scalability Limits

- **Practical limit:** ~5,000 events (memory constraint)
- **Optimization for larger datasets:** Batch processing with sliding windows
- **Parallel processing potential:** Similarity matrix is embarrassingly parallel

Section 10: Research Contributions

What Makes MITRE-CORE Research-Grade

1. Novel Algorithm

- First application of Union-Find to cybersecurity alert correlation
- Hybrid scoring combining network, identity, and temporal features
- Adaptive thresholding with theoretical foundation

2. Comprehensive Evaluation

- 7 baseline methods for fair comparison

- Statistical significance testing (not just accuracy scores)
- Synthetic dataset generation with quality validation

3. **Reproducibility**

- All algorithms deterministic (fixed random seeds)
- Complete code and documentation available
- MITRE ATT&CK framework provides standardized ground truth

4. **Theoretical Foundation**

- Parameters justified by literature (Valeur et al., 2004)
- Mathematical bounds on threshold values
- Proven correctness of Union-Find clustering

Key Technical Contributions

Contribution	Innovation	Impact
Union-Find Clustering	$O(\alpha(n))$ time, transitivity support	Correct grouping of multi-hop attacks
Adaptive Thresholding	Data-driven parameter selection	No manual tuning required
Multi-Modal Correlation	IP + hostname + temporal	Captures real attacker behavior
Stage Classification	ATT&CK-based severity	Prioritizes analyst attention
Comprehensive Evaluation	7 baselines + statistical tests	Research-grade validation

Summary

MITRE-CORE transforms raw security alerts into actionable attack intelligence through:

1. **Smart Clustering** — Union-Find algorithm groups related events in $O(n^2)$ time
2. **Adaptive Intelligence** — Thresholds adjust automatically based on data characteristics
3. **Attack Context** — MITRE ATT&CK mapping provides standardized threat language
4. **Visual Understanding** — Interactive dashboard makes complex chains understandable
5. **Research Rigor** — 7 baseline comparisons with statistical validation

For the 5-year-old: It's a superhero detective that never sleeps, watching all your toys, figuring out which ones the sneaky cookie thief touched, and telling you exactly how they stole the cookies! 🍪🕵️

Project Status: Phase 1 Complete  | Ready for Research Publication | MIT License